

Laboratorio 25

Sesión #25 Orquestación de microservicios para sistemas de IA escalables

Título del Laboratorio: Orquestación de Microservicios para Sistemas de IA Escalables

Duración: 2 horas

Objetivos del Laboratorio:

1. Configurar un entorno básico para la gestión de microservicios utilizando Kubernetes.
2. Desplegar un microservicio que sirve un modelo de IA para predicciones en tiempo real.
3. Implementar un balanceador de carga para manejar solicitudes concurrentes.
4. Aplicar un mecanismo de escalado automático basado en métricas de uso.
5. Reflexionar sobre los retos operativos en la orquestación de microservicios y proponer soluciones.

Materiales Necesarios:

1. **Software:**
 - a. Docker (para contenerización).
 - b. Kubernetes (Minikube o clúster gestionado).
 - c. Python 3.x con las siguientes librerías:
 - i. **Flask** (para crear la API del microservicio).
 - ii. **requests** (para pruebas de comunicación entre servicios).
 - iii. **pandas** y **scikit-learn** (para modelos de IA básicos).
2. **Entorno:**
 - a. Instalación de Docker y Kubernetes en la máquina local.
 - b. Configuración de Minikube para ejecutar un clúster local de Kubernetes.
3. **Dataset Público:**
 - a. [Iris Dataset](#) para la demostración de predicciones.

Documento de Ayuda:

1. *Documentación de Kubernetes*
2. *Docker Basics*
3. *Flask Documentation*
4. *Artículo: "Best Practices for Scaling Machine Learning Models in Production" (ACM Journal).*

Estructura del Laboratorio:

1. **Introducción a Kubernetes y Contenerización**
 - Breve explicación de la arquitectura y relevancia de Kubernetes.
 - Contexto sobre el uso de contenedores en microservicios.
2. **Actividad 1: Creación de un Microservicio de Inferencia con Python y Flask (50 minutos):**
 - Desarrollar un servicio básico que realiza predicciones utilizando un modelo entrenado.
 - Contenerizar el servicio utilizando Docker.
3. **Actividad 2: Despliegue del Microservicio en Kubernetes (70 minutos):**
 - Configurar un pod en Kubernetes para alojar el microservicio.
 - Implementar balanceo de carga mediante un **Service** de Kubernetes.
 - Configurar un escalado automático con Horizontal Pod Autoscaler (HPA).
4. **Conclusión y Reflexión**
 - Discusión sobre los aprendizajes clave y retos enfrentados durante el laboratorio.

Desarrollo de Actividades

Actividad 1: Creación de un Microservicio de Inferencia

Para ejecutar el código Python de la Actividad 1, es necesario instalar algunas librerías específicas y cumplir con los requisitos básicos del entorno. Aquí tienes los pasos detallados:

Librerías necesarias:

1. **Flask**: Para crear la API que servirá como microservicio.
2. **pandas**: Para manejar los datos que se usarán en las predicciones.
3. **scikit-learn**: Para entrenar y cargar el modelo de IA.
4. **pickle**: (Incluida en la biblioteca estándar de Python) para guardar y cargar el modelo entrenado.

Instalación de las librerías

Antes de ejecutar el código, asegúrate de tener instaladas estas librerías. Puedes hacerlo ejecutando el siguiente comando en la terminal:

```
pip install flask pandas scikit-learn
```

Verificar la instalación

Para confirmar que las librerías están correctamente instaladas, puedes ejecutar este script en Python:

```
try:

    import flask

    import pandas as pd

    from sklearn.ensemble import RandomForestClassifier

    print("Todas las librerías necesarias están instaladas
correctamente.")

except ImportError as e:

    print(f"Falta instalar una librería: {e}")
```

Si usas Docker

Si vas a contenerizar el microservicio (como en la Actividad 2), no necesitas instalar las librerías manualmente en tu máquina, ya que el Dockerfile incluye las instrucciones para instalar automáticamente las dependencias dentro del contenedor:

Dockerfile

Copiar código

```
# Dockerfile básico

FROM python:3.8-slim

WORKDIR /app

COPY . .

RUN pip install flask pandas scikit-learn

CMD ["python", "app.py"]
```

Al construir el contenedor con `docker build`, se instalarán todas las librerías en el entorno aislado del contenedor.

Paso 1: Entrenar un Modelo y Guardarlo Utilizar el Iris Dataset para entrenar un modelo de clasificación y guardarlo como un archivo

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
import pickle

# Cargar el dataset
df = pd.read_csv("https://raw.githubusercontent.com/mwaskom/seaborn-
data/master/iris.csv")

# Preprocesar los datos
X = df.drop("species", axis=1)
y = df["species"]

# Entrenar el modelo
model = RandomForestClassifier()
model.fit(X, y)

# Guardar el modelo
with open("model.pkl", "wb") as f:
    pickle.dump(model, f)

print("Modelo entrenado y guardado exitosamente.")
```

Paso 2: Crear una API Flask para Inferencia Desarrollar una API en Flask que cargue el modelo y realice predicciones.

```
from flask import Flask, request, jsonify
import pickle
import pandas as pd

# Cargar el modelo
with open("model.pkl", "rb") as f:
    model = pickle.load(f)

# Crear la API Flask
app = Flask(__name__)

@app.route("/predict", methods=["POST"])
def predict():
    data = request.json # Recibir datos en formato JSON
    df = pd.DataFrame(data)
```

```
predictions = model.predict(df)
return jsonify({"predictions": list(predictions)})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Paso 3: Contenerizar la API Crear un archivo **Dockerfile** para contenerizar el servicio.

```
# Usar una imagen base de Python
FROM python:3.8-slim

# Copiar los archivos al contenedor
WORKDIR /app
COPY . .

# Instalar las dependencias
RUN pip install flask pandas scikit-learn

# Exponer el puerto
EXPOSE 5000

# Comando para ejecutar la API
CMD ["python", "app.py"]
```

Construir y ejecutar el contenedor:

```
docker build -t microservice:latest .
docker run -p 5000:5000 microservice
```

Actividad 2: Despliegue del Microservicio en Kubernetes

El objetivo de esta actividad es desplegar el microservicio desarrollado en la **Actividad 1** en un clúster de Kubernetes, utilizando contenedores Docker y configurando un **Service** para exponer el microservicio a través de un balanceador de carga.

1. Configuración Previa

1. Requisitos:

- **Docker** instalado y configurado.
- **Kubernetes** instalado (puedes usar **Minikube** para un clúster local).
- Código del microservicio y el modelo guardado (**model.pkl**).

2. Dockerfile:

- Usa el Dockerfile desarrollado en la **Actividad 1** para contenerizar el microservicio.

Dockerfile

Copiar código

```
FROM python:3.8-slim
WORKDIR /app
COPY . .
RUN pip install flask pandas scikit-learn
EXPOSE 5000
CMD ["python", "app.py"]
```

3. Construcción y publicación de la imagen: Construye la imagen Docker y publícala en un registro de contenedores (puedes usar Docker Hub para este laboratorio).

bash

Copiar código

```
docker build -t <tu_usuario_docker>/microservice:latest .
docker push <tu_usuario_docker>/microservice:latest
```

2. Despliegue en Kubernetes

2.1. Archivo de configuración para el Deployment

Este archivo configura un Deployment de Kubernetes que creará y administrará los pods que ejecutan el microservicio.

Archivo: **deployment.yaml**

yaml

Copiar código

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: microservice-deployment
  labels:
    app: microservice
spec:
  replicas: 2 # Número inicial de réplicas
  selector:
    matchLabels:
      app: microservice
  template:
    metadata:
      labels:
```

```
    app: microservice
  spec:
    containers:
      - name: microservice
        image: <tu_usuario_docker>/microservice:latest # Imagen
del contenedor
        ports:
          - containerPort: 5000
```

Ejecuta el Deployment:

```
bash
Copiar código
kubectl apply -f deployment.yaml
```

Verifica que los pods se están ejecutando:

```
bash
Copiar código
kubectl get pods
```

2.2. Archivo de configuración para el Service

Crea un archivo que exponga el microservicio como un Service de Kubernetes.

Archivo: `service.yaml`

```
yaml
Copiar código
apiVersion: v1
kind: Service
metadata:
  name: microservice-service
spec:
  selector:
    app: microservice
  ports:
    - protocol: TCP
      port: 80          # Puerto que el servicio expone
      targetPort: 5000  # Puerto del contenedor
  type: LoadBalancer  # Balanceador de carga
```

Ejecuta el Service:

bash

Copiar código

```
kubectl apply -f service.yaml
```

Obtén la dirección IP asignada por el balanceador de carga (en Minikube, usa `minikube service`):

bash

Copiar código

```
minikube service microservice-service
```

3. Pruebas del Despliegue

Enviar una solicitud de prueba: Usa `curl` o una herramienta como **Postman** para enviar una solicitud al microservicio.

Ejemplo de solicitud con `curl`:

bash

Copiar código

```
curl -X POST -H "Content-Type: application/json" \
-d '["sepal_length": 5.1, "sepal_width": 3.5, "petal_length":
1.4, "petal_width": 0.2}]' \
http://<direccion_ip_del_service>/predict
```

Respuesta esperada:

json

Copiar código

```
{"predictions": ["setosa"]}
```

1.

Monitorear los Pods: Verifica el estado de los pods durante las pruebas.

bash

Copiar código

```
kubectl get pods -o wide
```

4. Escalado Automático con HPA

Configura un **Horizontal Pod Autoscaler (HPA)** para escalar dinámicamente los pods según la carga de CPU.

Archivo: `hpa.yaml`

yaml

Copiar código

```
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
  name: microservice-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: microservice-deployment
  minReplicas: 2
  maxReplicas: 5
  targetCPUUtilizationPercentage: 50
```

Aplica la configuración:

bash

Copiar código

```
kubectl apply -f hpa.yaml
```

Monitorea el escalado automático:

bash

Copiar código

```
kubectl get hpa
```

5. Conclusión

En esta actividad, se desplegó un microservicio de inferencia en un clúster de Kubernetes, utilizando un balanceador de carga para distribuir solicitudes y configurando un escalador automático para manejar cargas variables. Este flujo refleja cómo se implementan sistemas distribuidos de IA en entornos reales, garantizando escalabilidad y disponibilidad.

Reto del Laboratorio: Sesión 25 - Orquestación de Microservicios para Sistemas de IA Escalables

Descripción del Reto

El objetivo del reto es que los estudiantes implementen y optimicen un sistema de microservicios escalable para realizar inferencias de IA, simulando un entorno de alta demanda. Este reto consolidará los conocimientos adquiridos durante el laboratorio y les permitirá aplicar herramientas como Docker, Kubernetes e Istio para resolver un problema práctico.

Escenario del Reto

Un sistema de clasificación de imágenes basado en IA debe manejar solicitudes concurrentes provenientes de miles de usuarios. El desafío consiste en diseñar un entorno distribuido donde:

1. El microservicio de inferencia pueda escalar automáticamente con base en la carga del sistema.
2. Las solicitudes se distribuyan de manera eficiente entre múltiples instancias del microservicio.
3. Se implementen políticas de resiliencia y tolerancia a fallos.

Tareas del Reto

1. **Optimización del Sistema de Inferencia:**
 - Contenerizar un modelo de clasificación de imágenes ya entrenado (por ejemplo, basado en un modelo de Machine Learning como **sklearn** o **tensorflow**) y desplegarlo como un microservicio en Kubernetes.
2. **Implementar un Balanceador de Carga:**
 - Configurar un servicio en Kubernetes para distribuir el tráfico entre múltiples réplicas del microservicio.
3. **Configurar el Escalado Automático:**
 - Usar el Horizontal Pod Autoscaler (HPA) de Kubernetes para ajustar dinámicamente el número de instancias del microservicio en función de la CPU utilizada.
4. **Monitorear el Desempeño:**
 - Integrar herramientas de monitoreo (como **kubectl top** o Prometheus) para evaluar métricas del sistema, como el uso de CPU, latencia promedio y tasa de solicitudes.
5. **Añadir Resiliencia:**
 - Proponer y configurar políticas para manejar fallos en el microservicio, como reintentos automáticos o límites de tiempo para las solicitudes.

Criterios de Evaluación

1. **Funcionalidad:**
 - El sistema debe manejar solicitudes concurrentes de manera eficiente.
 - El escalado automático debe activarse bajo una alta carga.
2. **Eficiencia:**
 - Las métricas de desempeño (como latencia y uso de recursos) deben mantenerse dentro de rangos aceptables.
3. **Resiliencia:**
 - El sistema debe continuar funcionando incluso si una instancia del microservicio falla.

4. Documentación:

- Presentar un archivo **README** que describa los pasos para implementar el sistema, junto con la configuración de Kubernetes y cualquier herramienta adicional utilizada.

Entrega del Reto

Los estudiantes deben entregar:

1. Archivos de configuración de Kubernetes (**.yaml**) para los pods, servicios y HPA.
2. Código del microservicio de inferencia (archivo Python).
3. Evidencias del sistema en funcionamiento:
 - Capturas de pantalla del balanceador de carga distribuyendo tráfico.
 - Capturas de métricas del sistema bajo carga (CPU, latencia).
4. Un breve informe (máximo 2 páginas) explicando cómo diseñaron el sistema, los problemas encontrados y las soluciones implementadas.